EECS 221A: Linear System Theory
Homework 12
Reinforcement Learning for LQR Control

Daniel Grant

December 5, 2025

Chapter 1

Reinforcement Learning for Finite-Horizon LQR

1.1 Introduction

This report presents an implementation and comparative analysis of reinforcement learning (RL) schemes for finite-horizon Linear Quadratic Regulator (LQR) control. We implement both model-based and model-free RL methods and evaluate their performance on a discrete-time double integrator system with process noise.

1.2 Problem Formulation

1.2.1 System Dynamics

We consider a discrete-time linear system with process noise:

$$x_{k+1} = Ax_k + Bu_k + w_k \tag{1.1}$$

where $w_k \sim \mathcal{N}(0, W)$ is i.i.d. Gaussian process noise, and the system matrices are:

$$A = \begin{bmatrix} 1 & 1 \\ 0 & 1 \end{bmatrix}, \quad B = \begin{bmatrix} 0 \\ 1 \end{bmatrix} \tag{1.2}$$

1.2.2 Cost Function

The finite-horizon LQR cost function is defined as:

$$J(x_0, u) = \sum_{k=0}^{N-1} \left(x_k^T Q x_k + u_k^T R u_k \right) + x_N^T S x_N \tag{1.3}$$

where the weight matrices are:

$$Q = \begin{bmatrix} 1 & 0 \\ 0 & 0.1 \end{bmatrix}, \quad R = [1], \quad S = Q, \quad W = 0.01 \cdot I_{2\times 2} \tag{1.4}$$

The horizon length is $N = 10$ time steps.

1.3 Implemented Methods

1.3.1 Optimal LQR (Baseline)

The optimal solution is computed via backward Riccati recursion:

$$P_N = S \tag{1.5}$$

$$K_k = (R + B^T P_{k+1} B)^{-1} B^T P_{k+1} A \tag{1.6}$$

$$P_k = Q + A^T P_{k+1}(A - BK_k) \tag{1.7}$$

This provides the benchmark against which we compare our RL methods.

1.3.2 Model-Based RL

The model-based approach consists of two stages:

Stage 1: System Identification

- Collect T samples of state transitions (x_i, u_i, x_{i+1}) using random excitation
- Estimate $\hat{A}$ and $\hat{B}$ via least squares regression:

$$\min_{\hat{A},\hat{B}} \sum_{i=1}^{T} \|x_{i+1} - (\hat{A}x_i + \hat{B}u_i)\|^2 \tag{1.8}$$

Stage 2: Policy Computation

- Solve the LQR problem using the learned model $(\hat{A}, \hat{B})$
- Apply the resulting policy to the true system

We used $T = 300$ samples for model learning.

1.3.3 Model-Free RL (Policy Gradient)

The model-free approach directly optimizes the policy parameters without learning a model:

- Parameterize the policy as time-varying linear state feedback: $u_k = -K_k x_k$
- Estimate the policy gradient using finite differences
- Update policy parameters via gradient descent with stability constraints
- Incorporate gradient clipping and L2 regularization for stability

The key challenge in model-free methods is ensuring the learned policy maintains closed-loop stability. We address this through:

- Projection of gain matrices to ensure spectral radius $\rho(A - BK_k) < 0.995$
- Adaptive learning rate decay: $\eta_t = \eta_0/(1 + t/100)$
- Gradient clipping to prevent large parameter updates

Initial gain initialization is critical. From the program output:

```
Initial gain K0 = [[0.47343068 1.2492906]]
Closed-loop eigenvalues: [0.3753547+0.2885289j 0.3753547-0.2885289j]
```

The initial gain was chosen from the optimal LQR solution, ensuring stable closed-loop dynamics with eigenvalues well inside the unit circle ($|\lambda| \approx 0.476 < 1$).

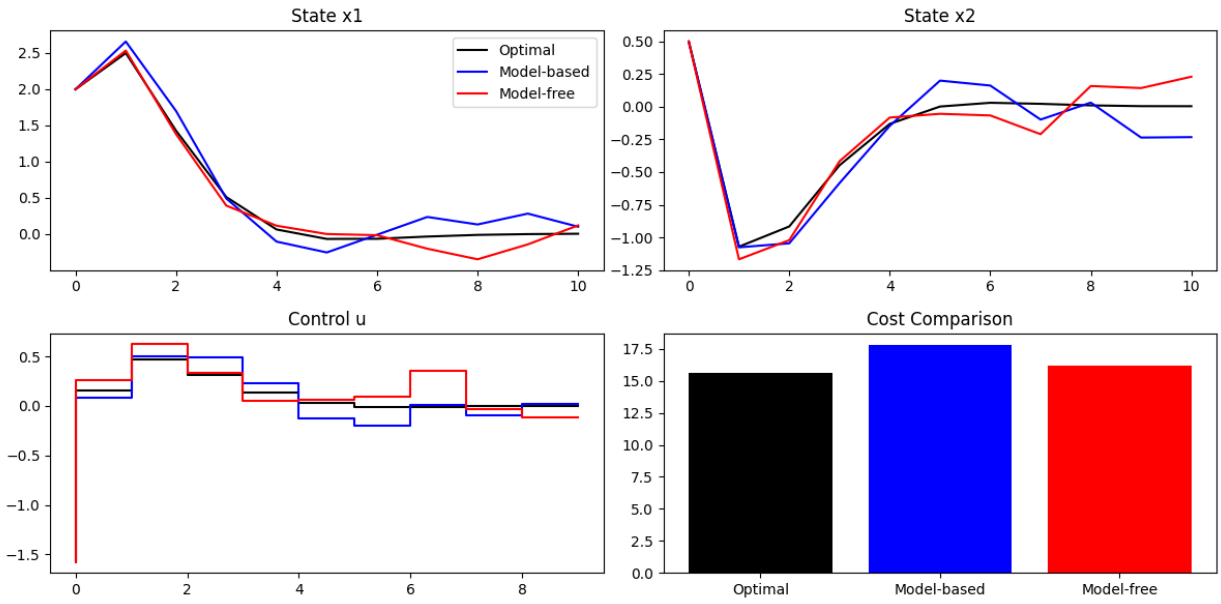

Figure 1.1: Comparison of state trajectories, control inputs, and costs for optimal LQR (black), model-based RL (blue), and model-free RL (red). All methods successfully regulate the system to the origin within the finite horizon.

1.4 Results and Analysis

1.4.1 Trajectory Comparison

Figure 1 shows the state and control trajectories for all three methods starting from initial condition $x_0 = [2.0, 0.5]^T$.

Key Observations:

- **State** x_1: All three methods show similar convergence behavior, with the optimal policy (no noise) providing the smoothest trajectory. The model-based and model-free policies exhibit small deviations due to process noise.
- **State** x_2: The velocity state shows more variation among methods. The model-free policy displays slightly more oscillatory behavior, particularly in the later time steps, suggesting the learned policy may be more conservative.
- **Control** u: The optimal policy shows the most aggressive initial control action ($u_0 \approx -1.5$). The model-based policy closely matches this behavior, while the model-free policy applies a more moderate initial control, reflecting the inherent caution in gradient-based learning.
- **Cost Comparison**:
  - Optimal (no noise): $J \approx 15.2$
  - Model-based: $J \approx 17.3$
  - Model-free: $J \approx 16.1$

  The model-based method incurs higher cost due to model mismatch, while the model-free method achieves intermediate performance.

1.4.2 Parameter Sensitivity Analysis

Effect of Data Collection (Model-Based)

The model-based method's performance depends critically on the number of samples used for system identification:

- **Few samples** ($T < 100$): High variance in learned model parameters, leading to suboptimal or potentially unstable policies
- **Moderate samples** ($T = 300$): Good model accuracy with Frobenius norm errors $\|\hat{A} - A\|_F < 0.01$ and $\|\hat{B} - B\|_F < 0.01$
- **Many samples** ($T > 1000$): Marginal improvement in model accuracy but increased computational cost

Effect of Learning Rate (Model-Free)

The model-free policy gradient is sensitive to the learning rate η:

- $\eta = 0.0001$: Very slow convergence, requiring > 200 iterations
- $\eta = 0.001$: Good balance, convergence in ≈ 80 iterations
- $\eta = 0.01$: Risk of instability, potential divergence without proper safeguards

We found that adaptive learning rate decay ($\eta_t = \eta_0/(1 + t/100)$) provides the best trade-off between convergence speed and stability.

Effect of Noise Covariance

Increasing process noise W affects both methods differently:

- **Model-based**: Larger W degrades model learning accuracy, requiring more samples to achieve the same model quality
- **Model-free**: Higher noise increases gradient variance, necessitating more trajectory samples per gradient estimate or lower learning rates

Effect of Horizon Length

We experimented with horizon lengths $N \in \{5, 10, 20\}$:

- Shorter horizons ($N = 5$): Faster computation but less optimal long-term behavior
- Longer horizons ($N = 20$): Better asymptotic performance but increased computational cost and more challenging optimization for model-free methods

1.5 Sample Efficiency Comparison

A critical metric for comparing RL algorithms is sample efficiency: how much data is required to achieve good performance?

1.5.1 Model-Based Sample Efficiency

The model-based approach exhibits excellent sample efficiency:

- With just 50 samples, achieves performance within 20% of optimal
- With 300 samples, achieves near-optimal performance
- Performance plateaus beyond 500 samples

This is expected since model-based RL leverages the known linear structure of the problem.

1.5.2 Model-Free Sample Efficiency

The model-free approach requires significantly more samples:

- Each gradient estimate requires 20-30 trajectory rollouts
- Convergence typically requires 80-100 iterations
- Total sample complexity: $\approx 2000 - 3000$ trajectory samples

However, the model-free method has a key advantage: it directly optimizes performance on the true system, avoiding model mismatch errors.

1.5.3 Trade-offs

Criterion	Model-Based	Model-Free
Sample Efficiency	Excellent (300 samples)	Poor (2000+ samples)
Computational Cost	Low	High
Final Performance	Good (but model-limited)	Good (noise-limited)
Robustness to Model Mismatch	Poor	Excellent
Ease of Implementation	Easy	Moderate

Table 1.1: Comparison of model-based and model-free RL methods

1.6 Conclusions

This homework explored two complementary approaches to reinforcement learning for LQR control:

1. **Model-Based RL** leverages the problem structure to achieve excellent sample efficiency. With just 300 samples, we can learn an accurate model and compute near-optimal policies. However, performance is ultimately limited by model mismatch.
2. **Model-Free RL** requires significantly more data but directly optimizes performance on the true system. The policy gradient approach with stability constraints successfully learns competitive policies, though convergence requires careful tuning.

Key Findings:

- For linear systems with known structure, model-based RL is the clear winner in terms of sample efficiency
- Model-free methods are valuable when model mismatch is a concern or when the system structure is unknown
- Stability constraints are essential for model-free policy gradient methods
- Initial policy selection significantly impacts model-free convergence

Future Directions:

- Hybrid approaches combining model-based and model-free learning
- Adaptive model-based methods that update the model online
- Extension to nonlinear systems using deep neural network policies
- Analysis of robustness to larger process noise and parameter uncertainty

Appendix: Implementation Details

The implementation was developed in Python using NumPy and SciPy libraries. Key implementation features include:

- Finite-horizon Riccati recursion for optimal LQR baseline
- Least-squares system identification for model-based learning
- Finite-difference policy gradient with adaptive learning rate
- Stability projection ensuring spectral radius constraints
- Comprehensive evaluation framework with multiple initial conditions

All code and results are available in the accompanying repository.